

```
In [ ]: %matplotlib inline
import matplotlib.pyplot as plt
import jax.numpy as jnp
import numpy as np
from jax import random, value_and_grad
import seaborn as snb

import bayesian
from bayesian import (
    # distributions
    gaussian_logpdf, gaussian_pdf, bernoulli_logpmf,
    mvn_logpdf, mvn_pdf, mvn_sample,
    gamma_logpdf, gamma_pdf, gamma_sample,
    beta_logpdf, beta_pdf, beta_sample,
    dirichlet_logpdf, dirichlet_pdf, dirichlet_sample,
    poisson_logpmf, poisson_pmf, poisson_sample,
    binomial_logpmf, binomial_pmf, binomial_sample,
    probit, sigmoid, softmax,
    # regression / Laplace
    compute_posterior_w, marginal_likelihood, laplace_approximation,
    # MCMC / HMC
    metropolis, HMC, leapfrog,
    compute_Rhat, compute_effective_sample_size,
    # Gaussian processes
    Hyperparameters, StationaryIsotropicKernel, Kernel,
    GaussianProcessRegression, optimize_marginal_likelihood,
    squared_exponential, matern12, matern32, generate_samples,
    # variational inference
    kl_gaussian, BlackBoxVI, VariationalGMM,
    # plotting
    plot_data, plot_contour, plot_heatmap,
    plot_trace, plot_mcmc_diagnostics, plot_posterior_1d, plot_predictions,
    plot_elbo, plot_vi_diagnostics, plot_with_uncertainty,
)

from jax import config
config.update("jax_enable_x64", True)

snb.set_style('darkgrid')
snb.set_theme(font_scale=1.25)
colors = snb.color_palette()
key = random.PRNGKey(0)
```

02477 Bayesian Machine Learning — Exam

Name _____

Student ID _____

Date _____

Written derivations are in the handwritten appendix (A–E). This notebook contains **computational** solutions only.

Part 1 — Conjugate Models

Question	Type	Location
Q1.1	Written derivation	Appendix A
Q1.2	Written derivation	Appendix A
Q1.3	Written derivation	Appendix A
Q1.4	Computational — plot prior / posterior	Below
Q1.5	Computational — posterior predictive	Below

```
In [ ]: # — Data / given values —————
# data = np.load('...')
# X, y = ...

# — Q1.4 — Plot prior and posterior —————
# Example: Beta-Binomial
# a0, b0 = 1.0, 1.0          # prior hyperparameters
# n_obs, x_obs = ...        # number of trials and successes
# a_post = a0 + x_obs
# b_post = b0 + (n_obs - x_obs)

theta = jnp.linspace(0, 1, 300)

fig, ax = plt.subplots(1, 1, figsize=(10, 4))
# ax.plot(theta, beta_pdf(theta, a0, b0), '--', label='Prior')
# ax.plot(theta, beta_pdf(theta, a_post, b_post), '-', label='Posterior')
# ax.axvline(x_obs / n_obs, color='k', ls=':', label='MLE')
ax.set(xlabel='$\\theta$', ylabel='Density', title='Q1.4 — Prior and Posterior')
ax.legend(); plt.tight_layout(); plt.show()

# — Q1.5 — Posterior predictive —————
# p(x_new = k | data) for k = 0, 1, ..., n_new
# n_new = ...
# k_vals = jnp.arange(n_new + 1)
# pp = jnp.array([jnp.exp(binomial_logpmf(k, n_new, p)) * beta_pdf(p, a_post
#                  for k in k_vals)]) # integrate numerically if needed

fig, ax = plt.subplots(1, 1, figsize=(10, 4))
# ax.bar(k_vals, pp / pp.sum(), label='Posterior predictive')
ax.set(xlabel='$k$', ylabel='Probability', title='Q1.5 — Posterior Predictive')
ax.legend(); plt.tight_layout(); plt.show()
```

Part 2 — Bayesian Linear Regression

Question	Type	Location
Q2.1	Written derivation (posterior)	Appendix B
Q2.2	Written derivation (marginal likelihood)	Appendix B
Q2.3	Written derivation	Appendix B
Q2.4	Computational — posterior predictions	Below
Q2.5	Computational — hyperparameter optimisation	Below

```
In [ ]: # — Data —————
# data = np.load('...')
# X_raw, y = data['X'], data['y'] # X_raw: (N,) or (N, D), y: (N,)

# — Design matrix (adapt basis to the problem) —————
# Polynomial: Phi = jnp.column_stack([X**k for k in range(M)])
# With bias: Phi = jnp.column_stack([jnp.ones(N), X])
# Phi shape must be (N, M)

# — Q2.4 — Posterior predictions —————
alpha, beta_prec = 1.0, 1.0 # prior precision, noise precision

m, S = compute_posterior_w(Phi, y, alpha, beta_prec)
print(f'Posterior mean: {np.array(m)}')
print(f'Posterior std: {np.sqrt(np.diag(np.array(S)))}')

x_star = jnp.linspace(X_raw.min(), X_raw.max(), 300)
# Phi_star = ... # design matrix for x_star

mu_f = Phi_star @ m
var_f = jnp.diag(Phi_star @ S @ Phi_star.T)
var_y = var_f + 1 / beta_prec

fig, ax = plt.subplots(1, 1, figsize=(12, 4))
ax.plot(x_star, mu_f, 'b-', label='Posterior mean')
ax.fill_between(x_star, mu_f - 2*jnp.sqrt(var_y), mu_f + 2*jnp.sqrt(var_y),
               alpha=0.25, color='b', label='95% predictive')
ax.scatter(X_raw.ravel(), y, c='k', s=15, zorder=5, label='Data')
ax.set(xlabel='$x$', ylabel='$y$', title='Q2.4 — Posterior predictive')
ax.legend(); plt.tight_layout(); plt.show()

# — Q2.5 — Hyperparameter optimisation —————
def neg_log_ml(log_theta):
    a, b = jnp.exp(log_theta[0]), jnp.exp(log_theta[1])
    return -marginal_likelihood(Phi, y, a, b)

from scipy.optimize import minimize
res = minimize(value_and_grad(neg_log_ml), jnp.zeros(2), jac=True)
alpha_opt, beta_opt = float(jnp.exp(res.x[0])), float(jnp.exp(res.x[1]))
print(f'Optimal alpha={alpha_opt:.4f}, beta={beta_opt:.4f}')
```

```
print(f'Log ML: {float(marginal_likelihood(Phi, y, alpha_opt, beta_opt)):.4
```

Part 3 — Gaussian Processes

Question	Type	Location
Q3.1	Written derivation (kernel properties)	Appendix C
Q3.2	Written — compute kernel matrix	Appendix C
Q3.3	Computational — GP regression	Below
Q3.4	Computational — hyperparameter optimisation	Below
Q3.5	Computational — compare kernel functions	Below

Q3.3 — GP Regression and posterior predictions

```
In [ ]: # — Data —————
# data = np.load('...')
# X_train shape: (N, 1), y_train shape: (N, 1)
# X_star shape: (P, 1) — dense prediction grid
# X_star = jnp.linspace(X_train.min(), X_train.max(), 300)[: , None]

# — Q3.3 — GP Regression —————
kernel = StationaryIsotropicKernel(squared_exponential)
hyper = Hyperparameters(kappa=1.0, lengthscale=1.0, sigma=0.3)

gp = GaussianProcessRegression(X_train, y_train, kernel, hyper)
mu, Sigma = gp.predict_y(X_star)
std = jnp.sqrt(jnp.diag(Sigma))

fig, ax = plt.subplots(1, 1, figsize=(12, 4))
ax.plot(X_star, mu, 'b-', label='Posterior mean')
ax.fill_between(X_star.ravel(), mu.ravel()-2*std, mu.ravel()+2*std,
               alpha=0.25, color='b', label='95% predictive')
ax.scatter(X_train.ravel(), y_train.ravel(), c='k', s=25, zorder=5, label='T
ax.set(xlabel='$x$', ylabel='$y$',
       title=f'Q3.3 — GP Regression ({hyper})')
ax.legend(); plt.tight_layout(); plt.show()

print(f'Log marginal likelihood: {float(gp.log_marginal_likelihood(hyper)):.4
```

Q3.4 — Hyperparameter optimisation via marginal likelihood

```
In [ ]: # — Q3.4 — Optimise hyperparameters via marginal likelihood —————
hyper_init = Hyperparameters(kappa=1.0, lengthscale=1.0, sigma=0.3)
hyper_opt = optimize_marginal_likelihood(gp, hyper_init, verbose=True)
gp_opt = GaussianProcessRegression(X_train, y_train, kernel, hyper_opt)
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 4), sharey=True)
for ax, (gp_i, label_i) in zip(axes, [
    (gp, f'Initial {hyper_init}'),
    (gp_opt, f'Optimised {hyper_opt}')],
    ):
    mu_i, Sig_i = gp_i.predict_y(X_star)
    std_i = jnp.sqrt(jnp.diag(Sig_i))
    ax.plot(X_star, mu_i, 'b-', label='Mean')
    ax.fill_between(X_star.ravel(), mu_i.ravel()-2*std_i, mu_i.ravel()+2*std_i,
                    alpha=0.25, color='b', label='95%')
    ax.scatter(X_train.ravel(), y_train.ravel(), c='k', s=20, zorder=5)
    ax.set(xlabel='$x$', ylabel='$y$', title=f'Q3.4 - {label_i}')
plt.tight_layout(); plt.show()

print(f'Log ML (init): {float(gp.log_marginal_likelihood(hyper_init)):.4f}')
print(f'Log ML (opt): {float(gp_opt.log_marginal_likelihood(hyper_opt)):.4f}')
```

Q3.5 — Compare kernel functions and hyperparameters

```
In [ ]: # — Q3.5 — Compare kernel functions —————
kernels = {
    'SE': StationaryIsotropicKernel(squared_exponential),
    'Matérn½': StationaryIsotropicKernel(matern12),
    'Matérn³/₂': StationaryIsotropicKernel(matern32),
}
hyper_init = Hyperparameters(kappa=1.0, lengthscale=1.0, sigma=0.3)

fig, axes = plt.subplots(1, len(kernels), figsize=(6*len(kernels), 4), share
for ax, (name, kernel) in zip(axes, kernels.items()):
    hyper_opt = optimize_marginal_likelihood(
        GaussianProcessRegression(X_train, y_train, kernel, hyper_init),
        hyper_init, verbose=False)
    gp_k = GaussianProcessRegression(X_train, y_train, kernel, hyper_opt)
    mu, Sigma = gp_k.predict_y(X_star)
    std = jnp.sqrt(jnp.diag(Sigma))
    ax.plot(X_star, mu, 'b-', label='Mean')
    ax.fill_between(X_star.ravel(), mu.ravel()-2*std, mu.ravel()+2*std,
                    alpha=0.25, color='b', label='95%')
    ax.scatter(X_train.ravel(), y_train.ravel(), c='k', s=20, zorder=5)
    ax.set(title=f'{name}\n{hyper_opt}', xlabel='$x$', ylabel='$y$')
plt.suptitle('Q3.5 — Kernel comparison', fontweight='bold')
plt.tight_layout(); plt.show()
```

Part 4 — MCMC and HMC Sampling

Question	Type	Location
Q4.1	Written derivation (detailed balance)	Appendix D
Q4.2	Written derivation (HMC algorithm)	Appendix D
Q4.3	Computational — Metropolis-Hastings	Below

	Question	Type	Location
	Q4.4	Computational — HMC	Below
	Q4.5	Computational — convergence diagnostics	Below

Q4.3 — Metropolis-Hastings sampler

```
In [ ]: # — Q4.3 — Metropolis-Hastings sampler —————
# Define log-target: log p(data | theta) + log p(theta)
def log_target(theta):
    # log-prior (e.g., standard normal)
    log_p = jnp.sum(gaussian_logpdf(theta, 0.0, 1.0))
    # log-likelihood
    # log_p += ...
    return log_p

D = ...      # number of parameters
tau = 0.5    # proposal std. dev. — aim for ~20-40% acceptance rate

samples_mh = metropolis(log_target, num_params=D, tau=tau,
                        num_iter=10_000, seed=0)
# samples_mh shape: (10_001, D)

warm_up = 1000
post_mh = samples_mh[warm_up:]

fig, ax = plt.subplots(1, 1, figsize=(10, 4))
plot_posterior_1d(ax, post_mh[:, 0], label='$\\theta_0$ (MH)')
ax.set_title('Q4.3 — MH posterior')
plt.tight_layout(); plt.show()

print(f'MH posterior mean: {np.array(post_mh.mean(0))}')
print(f'MH posterior std: {np.array(post_mh.std(0))}')
```

Q4.4 — Hamiltonian Monte Carlo (HMC)

```
In [ ]: # — Q4.4 — Hamiltonian Monte Carlo (HMC) —————
# Reuse log_target from Q4.3

theta0 = jnp.zeros(D)
samples = HMC(log_target, num_iterations=3000, theta0=theta0,
              num_leapfrog_steps=10, step_size=0.05, seed=0)
# samples shape: (3001, D)

warm_up = 500
post = samples[warm_up:]

fig, axes = plot_mcmc_diagnostics(samples[None, :, :], warm_up=warm_up,
                                param_names=[f'$\\theta_{i}$' for i in range(D)])
plt.tight_layout(); plt.show()

print(f'HMC posterior mean: {np.array(post.mean(0))}')
```

```
print(f'HMC posterior std: {np.array(post.std(0))}')

```

Q4.5 — Convergence diagnostics (R-hat, ESS)

```
In [ ]: # — Q4.5 — Convergence diagnostics —————
# Run a second chain with a different seed for R-hat / ESS
samples2 = HMC(log_target, num_iterations=3000, theta0=theta0,
               num_leapfrog_steps=10, step_size=0.05, seed=2, verbose=False)

# Stack chains: (num_chains, num_samples, num_params)
warm_up = 500
chains = jnp.stack([samples[None, :, :], samples2[None, :, :]], axis=0)

fig, axes = plot_mcmc_diagnostics(chains, warm_up=warm_up,
                                param_names=[f'$\\theta_{i}$' for i in range(D)])
plt.tight_layout(); plt.show()

Rhat = compute_Rhat(chains[:, warm_up:, :])
ESS = compute_effective_sample_size(chains[:, warm_up:, :])
print(f'R-hat: {np.array(Rhat)} (convergence if < 1.01)')
print(f'ESS: {np.array(ESS)}')
```

Part 5 — Variational Inference

Question	Type	Location
Q5.1	Written derivation (ELBO decomposition)	Appendix E
Q5.2	Written derivation (KL divergence)	Appendix E
Q5.3	Computational — BBVI	Below
Q5.4	Computational — ELBO / diagnostics	Below
Q5.5	Computational — compare VI to HMC	Below

Q5.3 — Black-Box Variational Inference

```
In [ ]: # — Q5.3 — Black-Box Variational Inference —————
# log_prior(w): w shape (S, D) → (S,)
def log_prior(w):
    return jnp.sum(gaussian_logpdf(w, 0.0, 1.0), axis=-1)

# log_lik(X, y, w): w shape (S, D) → (S,)
def log_lik(X, y, w):
    # example (logistic regression):
    # logits = w @ X.T # (S, N)
    # return jnp.sum(bernoulli_logpmf(y, sigmoid(logits)), axis=-1)
    raise NotImplementedError

D = ... # number of parameters
```

```

vi = BlackBoxVI(log_prior, log_lik, num_params=D,
                step_size=0.01, max_itt=2000, num_samples=20, seed=0)
vi.fit(X, y)

fig, ax = plt.subplots(figsize=(10, 3))
plot_elbo(ax, vi)
plt.tight_layout(); plt.show()

print(f'Final ELBO: {vi.ELBO_history[-1]:.3f}')

```

Q5.4 — ELBO convergence and variational parameter trajectories

```

In [ ]: # — Q5.4 — ELBO convergence and variational parameter trajectories —————
fig, axes = plot_vi_diagnostics(vi, param_names=[f'$\\theta_{i}$' for i in range(D)])
plt.suptitle('Q5.4 — VI diagnostics', fontweight='bold', y=1.01)
plt.tight_layout(); plt.show()

m_opt, v_opt = vi.unpack(vi.lam)
print('Optimal variational mean: ', np.array(m_opt))
print('Optimal variational std dev: ', np.sqrt(np.array(v_opt)))

# Posterior samples from q(w)
key, subkey = random.split(key)
vi_samples = vi.generate_posterior_samples(subkey, num_samples=2000)
print(f'Sample mean: {np.array(vi_samples.mean(0))}')
print(f'Sample std: {np.array(vi_samples.std(0))}')

```

Q5.5 — Compare VI posterior to MCMC

```

In [ ]: # — Q5.5 — Compare VI posterior to HMC —————
theta0 = vi.unpack(vi.lam)[0]
hmc_samples = HMC(log_target, num_observations=3000, theta0=theta0,
                  num_leapfrog_steps=10, step_size=0.05, seed=1, verbose=False)
hmc_post = hmc_samples[500:]

D = vi.D
fig, axes = plt.subplots(1, D, figsize=(5*D, 4), squeeze=False)
for i, ax in enumerate(axes[0]):
    m_vi, v_vi = vi.unpack(vi.lam)
    xs = jnp.linspace(float(hmc_post[:, i].min()) - 0.5,
                      float(hmc_post[:, i].max()) + 0.5, 300)
    ax.hist(np.array(hmc_post[:, i]), bins=40, density=True,
            alpha=0.5, color='b', label='HMC')
    ax.plot(xs, gaussian_pdf(xs, m_vi[i], jnp.sqrt(v_vi[i])),
            'r-', lw=2, label='VI (Gaussian)')
    ax.set(xlabel=f'$\\theta_{i}$', ylabel='Density')
    ax.legend()
plt.suptitle('Q5.5 — Variational posterior vs. HMC', fontweight='bold')
plt.tight_layout(); plt.show()

```